

# **INTERNSHIP REPORT**

## **VAULT OF CODES**

**Submitted by**

**V SHREE KAMALESH**

**211423244203**

*in partial fulfilment of the requirements  
for the award of the degree  
of*

**Bachelor of Technology**

**Department of Computer Science and Business Systems**



**PANIMALAR ENGINEERING COLLEGE**  
**(An Autonomous Institution, Affiliated to Anna University)**  
**Poonamallee, Chennai –600 123**

**OCTOBER – 2026**



## **DEPARTMENT OF COMPUTER SCIENCE AND BUSINESS SYSTEMS**

### **PANIMALAR ENGINEERING COLLEGE**

(An Autonomous Institution, Affiliated to Anna University)

Poonamallee, Chennai –600 123

### **BONAFIDE CERTIFICATE**

This is to certify that the internship report submitted by **V SHREE KAMALESH, 211423244203** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology of Anna University Chennai during the year 2026 - 2027.

**Signature of Head of the Department with  
date**

**Dr.D.Anuradha M.E., PGDBA, Ph.D.,**

Professor and Head,  
Department of Computer Science and Business  
Systems,  
Panimalar Engineering College,  
Chennai.

**Signature of the Supervisor with date**

**Dr. A. Anbarasa Pandian, M.E., Ph.D.,**

Associate Professor,  
Department of Computer Science and Business  
Systems,  
Panimalar Engineering College,  
Chennai.

Certified that the above candidate(s) was examined in the End Semester Internship Viva-Voce Examination held on \_\_\_\_\_

**INTERNAL EXAMINER- 1**

**INTERNAL EXAMINER- 2**

## **DECLARATION**

I affirm that the project given in this Internship Report of Vault of Codes being submitted in partial fulfilment of the requirements for the award of Bachelor of Technology is the original work carried out by me. It is not a part of any other report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Date:

Signature of the Student  
**V SHREE KAMALESH**

I certify that the declaration made by the above candidate is true to the best of my knowledge.

Date:

Signature of the Mentor  
**Dr. A. ANBARASA PANDIAN**

## ACKNOWLEDGEMENT

The satisfaction that one gets on completing an internship cannot be fully enjoyed without mentioning the people who made it possible.

I express my sincere thanks and grateful acknowledgment to our respected Secretary and Correspondent **Dr. P. CHINNADURAI, M.A., Ph.D.**, Panimalar Engineering College for providing the facilities in the college for undertaking my Internship Programme.

We would like to extend our heartfelt and sincere thanks to our Directors **Tmt. C. VIJAYARAJESWARI, Dr. C. SAKTHI KUMAR, M.E., Ph.D.** and **Dr. SARANYA SREE SAKTHI KUMAR, B.E., M.B.A., Ph.D.** for providing us with the necessary facilities for completion of this internship.

We also express our gratitude to our Principal **Dr. K. MANI, M.E., Ph.D.**, for his timely concern and encouragement provided to us throughout the internship.

I wish to express my sense of gratitude to **Dr. D. ANURADHA, M.E., PGDBA, Ph.D.**, Head of the Department of Computer Science and Business Systems for her constant support throughout my study.

I solemnly express my sense of gratitude to my Mentor **Dr. A. ANBARASA PANDIAN, M.E., Ph.D.**, Associate Professor for his encouragement, constant help, valuable guidance, suggestions and blessings which contributed immensely for the successful completion of the Internship Programme.

I would like to thank the other faculty members of Department of Computer Science and Business Systems, Panimalar Engineering College for their support and motivation throughout this internship.

I wish to thank my parents who always believed in me and had faith in me in whatever I wished to do and last but not the least I would thank my friends who have helped me endlessly.

## TABLE OF CONTENTS

| SL NO | TITLE                    | PAGE NO |
|-------|--------------------------|---------|
| 1     | ABSTRACT                 | 1       |
| 2     | COMPANY PROFILE          | 2       |
| 3     | COMPANY CERTIFICATE      | 4       |
| 4     | INTRODUCTION             | 5       |
| 5     | EXISTING SYSTEM          | 7       |
| 6     | PROPOSED METHODOLOGY     | 8       |
| 7     | IMPLEMENTATION & RESULTS | 14      |
| 8     | CONCLUSION               | 26      |

# **CHAPTER 1**

## **ABSTRACT**

In today's fast-paced academic and professional environment, students and working professionals handle a large number of tasks with different deadlines and priorities. Remembering these tasks mentally or noting them on paper often leads to missed deadlines, poor prioritisation and reduced productivity. A simple, accessible and well-designed digital tool can help people organise their work effectively.

This report presents the work carried out during a one-month Web Development internship at Vault of Codes from 01 June 2025 to 01 July 2025. The main project undertaken during the internship was the design and development of TaskFlow – a responsive Task Manager web application built using HTML5, CSS3 and vanilla JavaScript, without any external frameworks or libraries.

TaskFlow allows users to add, edit, complete and delete tasks, assign a due date, a priority level (High, Medium or Low) and a category to each task, and instantly search, filter and sort the task list. A statistics panel with a progress bar shows the total, pending, completed and overdue tasks at a glance, and overdue tasks are automatically highlighted. All data is stored in the browser using the Web Storage (localStorage) API, so tasks remain available even after the page is closed or refreshed. The application also supports a dark mode and a fully responsive layout that adapts to desktop, tablet and mobile screens.

The application was tested using 15 functional test cases covering task creation, validation, editing, deletion, filtering, searching, sorting, persistence, theme switching and responsiveness, and all the test cases passed successfully. The project provided hands-on experience in front-end development, DOM manipulation, event handling, responsive design and client-side data storage.

## **CHAPTER 2**

### **COMPANY PROFILE**

#### **2.1 About the Organisation**

Vault of Codes (VaultofCodes.in) is an Indian ed-tech organisation that provides online training programmes and internships in software development and related technology domains. The organisation focuses on helping students gain practical, industry-relevant skills by working on real projects under a structured internship programme. Vault of Codes is registered with the All India Council for Technical Education (AICTE) as a corporate partner for internships and is a Google for Education Partner.

#### **2.2 Vision and Mission**

The vision of Vault of Codes is to make quality technical education and practical exposure accessible to every student. Its mission is to bridge the gap between classroom learning and industry requirements by offering hands-on, project-based internships that build confidence, technical competence and employability among students.

#### **2.3 Internship Domains Offered**

Vault of Codes offers internships in a number of technology domains, including:

- Web Development
- Python Programming
- Java Programming
- C and C++ Programming
- Artificial Intelligence and Machine Learning
- Data Science and Analytics
- Android Application Development

#### **2.4 Internship Model**

The internship is conducted in a remote mode. Each intern is given a set of tasks related to the chosen domain, which must be completed within the internship duration. Interns develop the projects independently, maintain their source code in a GitHub repository and submit their work for evaluation. On successful completion of the assigned tasks, the intern is awarded a Certificate of Internship.

## 2.5 Internship Details

| Particular         | Details                                |
|--------------------|----------------------------------------|
| Name of the Intern | V SHREE KAMALESH                       |
| Organisation       | Vault of Codes (VaultofCodes.in)       |
| Domain             | Web Development                        |
| Designation        | Web Development Intern                 |
| Duration           | 01 June 2025 to 01 July 2025 (1 month) |
| Mode               | Remote                                 |
| AICTE Corporate ID | CORPORATE6511252d7c3271695622445       |

*Table 2.1 Internship Details*

## 2.6 Roles and Responsibilities of the Intern

As a Web Development Intern, the following responsibilities were carried out during the internship period:

- Understanding the requirements of the assigned task and planning the features of the application before starting development.
- Designing the user interface and page layout, keeping usability and responsiveness in mind.
- Developing the application using HTML5, CSS3 and JavaScript and following clean, readable coding practices.
- Maintaining the source code in a GitHub repository with regular commits throughout the development period.
- Testing the application in different browsers and screen sizes, identifying bugs and fixing them.
- Documenting the work and submitting the completed project for evaluation within the internship deadline.

## CHAPTER 3

### COMPANY CERTIFICATE



Figure 3.1 Certificate of Internship issued by Vault of Codes

## **CHAPTER 4**

### **INTRODUCTION**

#### **4.1 Overview**

Web development is the process of building applications that run in a web browser. Modern web applications are built using three core technologies: HTML (HyperText Markup Language), which defines the structure and content of a page; CSS (Cascading Style Sheets), which controls the layout, colours and visual presentation; and JavaScript, which adds interactivity and logic. Because web applications run in any browser, they can be accessed on desktops, laptops, tablets and smartphones without installation.

Task management applications are among the most widely used productivity tools. They help users record what needs to be done, decide what is most important and track their progress. Building one is an excellent way to learn front-end fundamentals such as forms, user interaction, data storage and responsive design.

#### **4.2 Problem Statement**

Students and professionals frequently lose track of their tasks and deadlines because they rely on memory, paper notes or scattered reminders. Many existing productivity applications are complex, require sign-up and an internet connection, or are cluttered with features that are not needed for everyday use. There is a need for a lightweight, fast and easy-to-use web application that lets users quickly record tasks, set priorities and deadlines, and see clearly what is pending and what is overdue – on any device.

#### **4.3 Objectives**

1. To design a clean and intuitive user interface for managing daily tasks.
2. To implement the core operations of adding, editing, completing and deleting tasks.
3. To allow users to assign a due date, priority and category to each task.
4. To provide search, filtering and sorting so that tasks can be found quickly.
5. To store tasks persistently in the browser using the localStorage API.
6. To build a responsive layout that works on desktop and mobile screens, with a dark mode option.

#### **4.4 Scope of the Project**

TaskFlow is a single-user, client-side application that runs entirely in the browser and does not require a server or an internet connection after it is loaded. It is suitable for personal task management by students and professionals. The design can be extended in future with user accounts, cloud synchronisation and reminders.

## 4.5 Tools and Technologies Used

| Category        | Tool / Technology                                  |
|-----------------|----------------------------------------------------|
| Structure       | HTML5                                              |
| Styling         | CSS3 (Flexbox, Grid, Media Queries, CSS Variables) |
| Logic           | JavaScript (ES6)                                   |
| Data Storage    | Web Storage API (localStorage)                     |
| Code Editor     | Visual Studio Code                                 |
| Testing         | Google Chrome / Microsoft Edge Developer Tools     |
| Version Control | Git and GitHub                                     |

*Table 4.1 Tools and Technologies*

## 4.6 Overview of Core Technologies

### 4.6.1 HTML5

HTML5 is the latest version of the HyperText Markup Language and is used to define the structure of a web page. It introduced semantic elements such as header, main, section and footer, which improve readability and accessibility. HTML5 also provides input types such as date and attributes such as maxlength.

### 4.6.2 CSS3

CSS3 is used to style the elements defined in HTML. Flexbox, a one-dimensional layout model, is used for the form controls and the toolbar, while CSS Grid, a two-dimensional layout model, arranges the statistics cards. Media queries apply different styles based on screen width, which is the basis of responsive design. CSS custom properties (variables) store values such as colours in one place, so the entire colour scheme can be changed by redefining a few variables – a feature used to implement the dark mode.

### 4.6.3 JavaScript (ES6)

JavaScript makes the page interactive. ES6 (ECMAScript 2015) introduced features such as const, arrow functions, template literals and the spread operator, which are used throughout TaskFlow along with array methods like filter, find and sort. JavaScript interacts with the page through the Document Object Model (DOM), a tree representation of the page that can be modified at run time.

### 4.6.4 Web Storage API (localStorage)

The Web Storage API lets a website store data in the browser. Data in localStorage does not expire, and objects are saved as JSON text using JSON.stringify() and read back with JSON.parse().

## CHAPTER 5

### EXISTING SYSTEM

#### 5.1 Description of the Existing System

In the existing approach, most students manage their tasks using paper diaries, sticky notes, the notes application on their phone or reminders sent through messaging applications. These methods are simple, but they do not provide any way of prioritising tasks, sorting them by deadline or seeing an overview of progress.

Full-featured productivity platforms are also available, but they usually require the user to create an account, depend on an internet connection and offer a large number of features such as team collaboration, integrations and paid plans. For a student who simply wants to keep track of personal tasks, these platforms can be heavy and time-consuming to learn.

#### 5.2 Limitations of the Existing System

- **No prioritisation:** Paper notes and plain text lists do not allow tasks to be ranked by importance.
- **No deadline tracking:** Overdue tasks are not highlighted, so deadlines are easily missed.
- **Difficult to search:** Finding a particular task in a long list of notes is slow.
- **No progress overview:** Users cannot see at a glance how many tasks are pending or completed.
- **Complexity:** Many digital tools require sign-up, an internet connection and time to learn.
- **Poor mobile experience:** Some simple web tools are not designed for small screens.

#### 5.3 Comparison of Existing Approaches

Table 5.1 compares the common approaches for managing personal tasks with the proposed TaskFlow application.

| Feature               | Paper Notes | Phone Notes App | Full-featured Platforms | TaskFlow |
|-----------------------|-------------|-----------------|-------------------------|----------|
| Priority levels       | No          | No              | Yes                     | Yes      |
| Due date tracking     | Manual      | Limited         | Yes                     | Yes      |
| Overdue highlighting  | No          | No              | Yes                     | Yes      |
| Search and filter     | No          | Basic           | Yes                     | Yes      |
| Progress overview     | No          | No              | Yes                     | Yes      |
| Works without sign-up | Yes         | Yes             | No                      | Yes      |
| Works offline         | Yes         | Yes             | Limited                 | Yes      |

*Table 5.1 Comparison of Existing Approaches with TaskFlow*

## CHAPTER 6

### PROPOSED METHODOLOGY

#### 6.1 Proposed System

The proposed system, TaskFlow, is a lightweight and responsive task manager web application. It provides a single-page interface in which users can create tasks with a title, due date, priority and category, mark them as completed, edit or delete them, and quickly search, filter and sort the list. A statistics panel and progress bar summarise the user's progress, and overdue tasks are highlighted automatically. All data is saved in the browser's localStorage, so no account or server is required.

#### 6.2 Advantages of the Proposed System

- Simple and intuitive interface that requires no sign-up or installation.
- Tasks can be prioritised and organised by category and due date.
- Overdue tasks are highlighted automatically, reducing missed deadlines.
- Instant search, filtering and sorting of tasks.
- Data persists across browser sessions using localStorage.
- Responsive design and dark mode for comfortable use on any device.

#### 6.3 Modules

- **Task Input Module:** A form to enter the task title, due date, priority and category, with validation to prevent empty tasks.
- **Task List Module:** Displays each task with its details, a checkbox to mark completion and buttons to edit or delete it. Priority is indicated by a coloured left border.
- **Search, Filter and Sort Module:** Allows tasks to be searched by title or category, filtered by status (All, Pending, Completed) and sorted by creation time, due date or priority.
- **Statistics Module:** Shows the total, pending, completed and overdue task counts and a progress bar of the percentage of tasks completed.
- **Storage Module:** Saves and loads tasks and the theme preference using the localStorage API.
- **Theme Module:** Switches between light and dark modes using CSS custom properties.

#### 6.4 Requirement Analysis

Before development began, the requirements of the application were identified and divided into functional requirements, which describe what the system must do, and non-functional requirements, which describe how well the system must perform.

#### 6.4.1 Functional Requirements

| ID  | Requirement                                                                         |
|-----|-------------------------------------------------------------------------------------|
| FR1 | The user shall be able to add a task with a title, due date, priority and category. |
| FR2 | The system shall not allow a task to be added with an empty title.                  |
| FR3 | The user shall be able to edit, delete and mark a task as completed.                |
| FR4 | The user shall be able to search tasks by title or category.                        |
| FR5 | The user shall be able to filter tasks by status and sort them by date or priority. |
| FR6 | The system shall highlight overdue tasks and display task statistics.               |
| FR7 | The system shall save all tasks and restore them when the page is reopened.         |
| FR8 | The user shall be able to switch between light and dark themes.                     |

*Table 6.1 Functional Requirements*

#### 6.4.2 Non-Functional Requirements

| Requirement     | Description                                                                       |
|-----------------|-----------------------------------------------------------------------------------|
| Usability       | The interface should be simple enough to use without any instructions.            |
| Performance     | Every action should update the screen instantly without reloading the page.       |
| Reliability     | The application should keep working even if stored data is corrupted.             |
| Portability     | The application should run in any modern browser on any operating system.         |
| Security        | Text entered by the user must not be executed as HTML or script.                  |
| Maintainability | Structure, style and logic should be kept in separate files with small functions. |

*Table 6.2 Non-Functional Requirements*

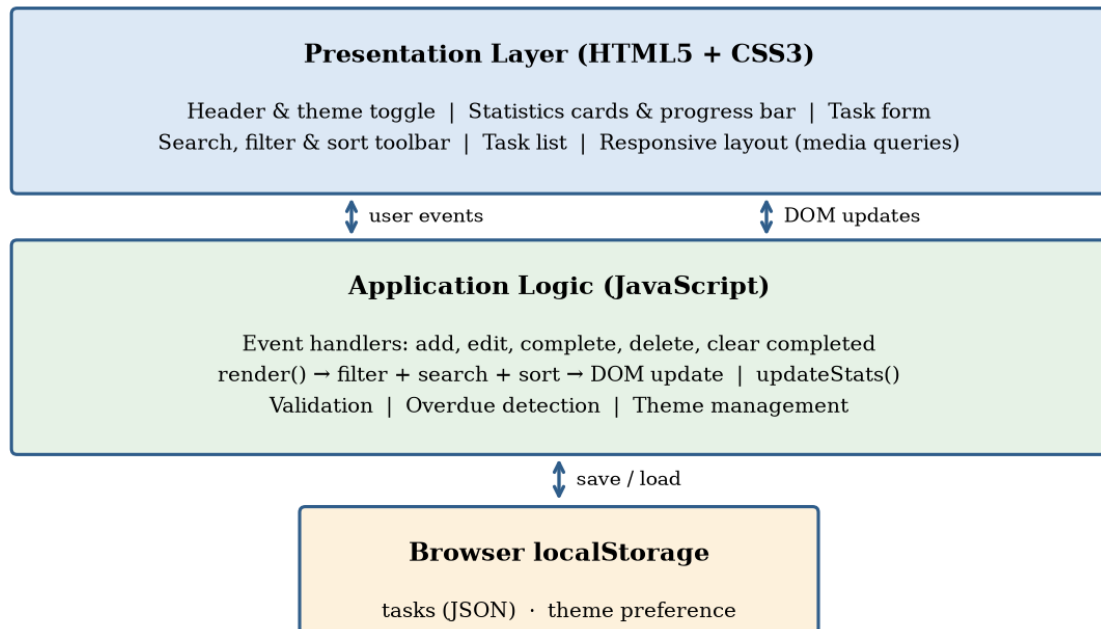
#### 6.4.3 Hardware and Software Requirements

| Component         | Requirement                                        |
|-------------------|----------------------------------------------------|
| Processor         | Any dual-core processor or above                   |
| RAM               | 2 GB or above                                      |
| Display           | Any screen of width 360 pixels or above            |
| Operating System  | Windows, Linux, macOS, Android or iOS              |
| Web Browser       | Any modern browser (Google Chrome, Microsoft Edge) |
| Development Tools | Visual Studio Code, Git and GitHub                 |

*Table 6.3 Hardware and Software Requirements*

## 6.5 System Architecture

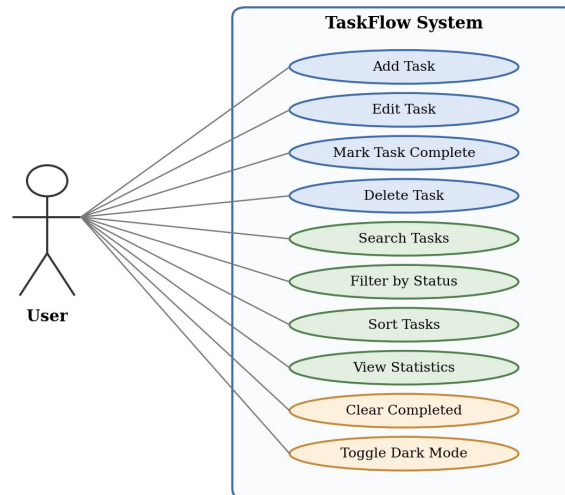
The application follows a simple three-layer client-side architecture, shown in Figure 6.1. The presentation layer (HTML and CSS) displays the interface. The application logic layer (JavaScript) responds to user events, updates the list of tasks in memory and re-renders the interface. The storage layer (localStorage) saves the tasks and the theme preference as JSON so that they are restored the next time the page is opened.



**Figure 6.1 System Architecture of TaskFlow**

## 6.6 Use Case Diagram

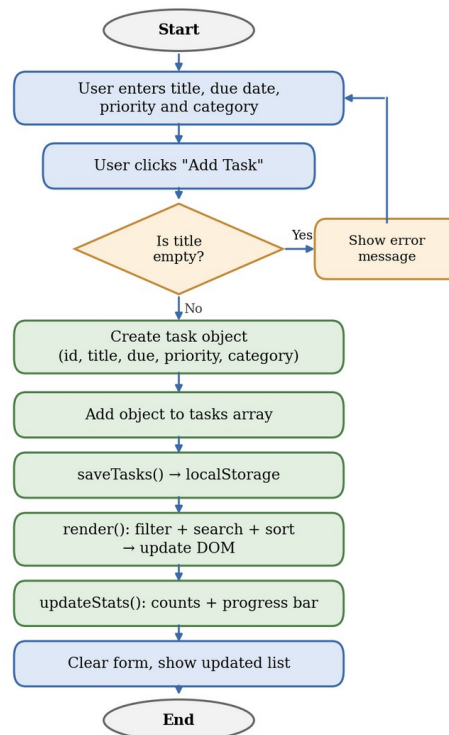
The use case diagram in Figure 6.2 shows the interactions between the user and the system. TaskFlow has a single actor, the User, who can perform all the operations of the application. The use cases are grouped into task operations (add, edit, complete and delete), organisation and viewing operations (search, filter, sort and view statistics) and utility operations (clear completed tasks and toggle the theme).



**Figure 6.2 Use Case Diagram of TaskFlow**

### 6.7 Workflow of Adding a Task

Figure 6.3 shows the sequence of steps that take place when the user adds a new task. When the form is submitted, the title is first validated. If the title is empty, an error message is shown and the user is asked to enter it again. Otherwise, a new task object is created and added to the tasks array, the array is saved to localStorage, the visible list is re-rendered after applying the current filter, search and sort, and the statistics panel is updated. The same save-render-update cycle is followed for every other operation, such as editing, completing and deleting tasks, which keeps the interface and the stored data always in sync.



**Figure 6.3 Workflow of Adding a Task**

## 6.8 Data Design

Each task is stored as a JavaScript object, and the complete list of tasks is stored in localStorage as a JSON array under the key taskflow.tasks. The fields of a task object are described in Table 6.4.

| S.No | Field    | Type    | Description                                     |
|------|----------|---------|-------------------------------------------------|
| 1    | id       | Number  | Unique identifier (timestamp at creation)       |
| 2    | title    | String  | Description of the task (maximum 80 characters) |
| 3    | due      | String  | Due date in YYYY-MM-DD format (optional)        |
| 4    | priority | String  | high, medium or low                             |
| 5    | category | String  | Personal, Work, Study or Shopping               |
| 6    | done     | Boolean | Whether the task is completed                   |
| 7    | created  | Number  | Creation time, used for sorting                 |

*Table 6.4 Structure of a Task Object*

The due date is stored as a YYYY-MM-DD string so that dates can be compared and sorted directly as text, without converting them into Date objects.

## 6.9 User Interface Design

The interface was first planned as a simple wireframe and then built as a single page divided into the following sections, arranged from top to bottom:

- **Header:** Shows the application name and the theme toggle button.
- **Statistics panel:** Four cards showing the total, pending, completed and overdue tasks, followed by a progress bar.
- **Task form:** Fields for the task title, due date, priority and category, and the Add Task button.
- **Toolbar:** The search box, the All / Pending / Completed filter buttons and the sort dropdown.
- **Task list:** The list of tasks with a checkbox, details, and edit and delete buttons, along with the number of tasks left and the Clear completed option.

Colours are used consistently to convey meaning, so that the user can understand the state of a task without reading every detail. The colour coding used in the interface is listed in Table 6.5.

| Element               | Colour | Meaning           |
|-----------------------|--------|-------------------|
| Left border of a task | Red    | High priority     |
| Left border of a task | Orange | Medium priority   |
| Left border of a task | Green  | Low priority      |
| Due date text         | Red    | Task is overdue   |
| Struck-through title  | Grey   | Task is completed |
| Buttons and header    | Blue   | Primary actions   |

*Table 6.5 Colour Coding Used in the Interface*

### 6.10 Development Methodology

The project was developed incrementally over the four weeks of the internship, as summarised in Table 6.6. Each week added a working layer on top of the previous one.

| Week   | Activities                                                                 |
|--------|----------------------------------------------------------------------------|
| Week 1 | Requirement analysis, study of HTML5 and CSS3, UI wireframe design         |
| Week 2 | HTML structure, CSS styling with Flexbox and Grid, responsive layout       |
| Week 3 | JavaScript logic for add, edit, delete, complete, search, filter and sort  |
| Week 4 | localStorage persistence, dark mode, testing, bug fixing and documentation |

*Table 6.6 Weekly Plan of Work*

In the first week, the requirements were listed and the layout of the page was sketched. In the second week, the static page was built and styled so that it looked correct on both large and small screens. In the third week, the JavaScript logic was written to make every control functional. In the final week, persistence, dark mode, testing and documentation were completed.

## CHAPTER 7

# IMPLEMENTATION & RESULTS

### 7.1 Implementation

The application consists of three files: `index.html`, which defines the page structure; `style.css`, which contains all the styling including the light and dark themes and the responsive media queries; and `script.js`, which contains the application logic. The JavaScript code is organised into small functions, each responsible for one job – loading data, saving data, rendering the list, updating statistics or handling a particular user action. Every user action follows the same cycle: the tasks array is updated, the array is saved to `localStorage`, and the interface is re-rendered. The key parts of the implementation are described below.

#### 7.1.1 Application State and Initialisation

The complete state of the application is held in a few variables at the top of `script.js`: the array of tasks, the currently selected status filter and the id of the task being edited, if any. Constants are used for the `localStorage` keys and for the ranking of priorities used while sorting. A small helper function `$` is used as a shortcut for `document.querySelector`. When the page loads, the saved theme is applied and the task list is rendered for the first time.

#### Listing 7.1 Application state and initialisation

```
const STORAGE_KEY = "taskflow.tasks";
const THEME_KEY = "taskflow.theme";
const PRIORITY_RANK = { high: 0, medium: 1, low: 2 };
const $ = (sel) => document.querySelector(sel);

let tasks = loadTasks(); // array of task objects
let filter = "all";      // "all" | "active" | "completed"
let editingId = null;    // id of the task being edited

function init() {
  applyTheme(loadTheme());
  render();
}
init();
```

#### 7.1.2 Persistence using `localStorage`

Tasks are loaded from `localStorage` when the page opens and saved after every change. The code is wrapped in `try/catch` blocks so that the application continues to work even if stored data is corrupted or storage is unavailable. When no saved data is found, for example on the very first visit, a small set of sample tasks is returned so that the user can immediately see how the application works.

## Listing 7.2 Loading and saving tasks

```
function loadTasks() {
  try {
    const saved = JSON.parse(localStorage.getItem(STORAGE_KEY));
    if (Array.isArray(saved)) return saved;
  } catch (e) { /* ignore corrupt data */ }
  return sampleTasks();
}

function saveTasks() {
  try { localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks)); }
  catch (e) { /* storage unavailable */ }
}
```

### 7.1.3 Rendering the Task List

The `render()` function is responsible for drawing the task list on the screen. It obtains the visible tasks after filtering, searching and sorting, converts each task into an HTML list item using a template literal, and inserts the result into the task list in a single operation. If no task matches the current filter or search, an empty-state message is shown instead. The function also updates the number of tasks left and calls `updateStats()` so that the statistics panel always matches the list.

Each list item carries the task's id in a `data-id` attribute and its priority as a CSS class, which is used to draw the coloured left border. The title is passed through `escapeHtml()` before being inserted, and a helper function `formatDate()` converts the stored YYYY-MM-DD date into a readable form such as "22 Jun 2025" using `toLocaleDateString()`.

### Listing 7.3 Rendering the task list

```
function render() {
  const list = visibleTasks();
  $("#taskList").innerHTML = list.length
    ? list.map(taskTemplate).join("")
    : `<li class="empty">No tasks to show.</li>`;
  $("#tasksLeft").textContent =
    `${tasks.filter(t => !t.done).length} tasks left`;
  updateStats();
}

function taskTemplate(t) {
  const late = isOverdue(t);
  return `
<li class="task ${t.priority} ${t.done ? "done" : ""}" data-id="${t.id}">
  <input type="checkbox" class="check" ${t.done ? "checked" : ""}>
  <div class="info">
    <span class="title">${escapeHtml(t.title)}</span>
    <span class="meta ${late ? "overdue" : ""}">
      ${formatDate(t.due)}${late ? " - Overdue" : ""}</span>
    <span class="tag">${t.category}</span>
    <span class="tag">${t.priority}</span>
  </div>
  <button class="edit" title="Edit"></button>
  <button class="delete" title="Delete"></button>
</li>`;
}

function formatDate(iso) {
  if (!iso) return "No due date";
  return new Date(iso + "T00:00").toLocaleDateString("en-GB",
    { day: "numeric", month: "short", year: "numeric" });
}
```

#### 7.1.4 Adding and Editing a Task

The same form is used both for adding a new task and for editing an existing one. When the form is submitted, the title is validated first. If a task is being edited, its fields are updated; otherwise a new task object is created and added to the beginning of the list so that it appears at the top.

### Listing 7.4 Handling form submission

```
form.addEventListener("submit", (e) => {
  e.preventDefault();
  const title = ($("#title").value.trim());
  if (!title) { showError("Please enter a task title."); return; }
  const data = { title, due: ($("#due").value,
    priority: ($("#priority").value,
    category: ($("#category").value );
  if (editingId) {
    Object.assign(tasks.find((t) => t.id === editingId), data);
    editingId = null;
  } else {
    tasks.unshift({ id: Date.now(), ...data,
      done: false, created: Date.now() });
  }
  saveTasks(); render(); form.reset();
});
```

When the user clicks the edit button of a task, the `startEdit()` function copies the task's values back into the form fields and remembers its id in `editingId`. The form is scrolled into view and the title field is focused, so the user can make changes and save them using the same button. Reusing one form for both operations keeps the interface simple and avoids duplicating the validation logic.

### 7.1.5 Search, Filter and Sort

Before rendering, the list of tasks is passed through the status filter, the search query and the selected sort order. Only the resulting tasks are displayed. The original tasks array is never modified by this process, so changing the filter or clearing the search immediately brings back the full list.

#### Listing 7.5 Search, filter and sort logic

```
function visibleTasks() {
  const q = $("#search").value.trim().toLowerCase();
  const sort = $("#sortBy").value;
  return tasks
    .filter((t) => (filter === "active" ? !t.done
      : filter === "completed" ? t.done : true))
    .filter((t) => t.title.toLowerCase().includes(q) ||
      t.category.toLowerCase().includes(q))
    .sort((a, b) => {
      if (sort === "due")
        return (a.due || "9999").localeCompare(b.due || "9999");
      if (sort === "priority")
        return PRIORITY_RANK[a.priority] - PRIORITY_RANK[b.priority];
      return b.created - a.created;
    });
}
```

Tasks without a due date are given the placeholder value “9999” while sorting, so that they appear at the end of the list when sorting by due date. The search box calls `render()` on every keystroke, giving instant results, while the filter buttons update the filter variable and highlight the selected button before re-rendering.

### 7.1.6 Event Delegation and Input Escaping

Instead of attaching separate event listeners to every task, a single click listener is attached to the task list. It identifies the clicked task using the `data-id` attribute and performs the required action (complete, edit or delete). This keeps the code efficient even when the list contains many tasks, and it continues to work for tasks that are added later because the listener belongs to the list rather than to individual items. User-entered text is escaped before being inserted into the page to prevent HTML injection.

### Listing 7.6 Event delegation and HTML escaping

```
$("#taskList").addEventListener("click", (e) => {
  const item = e.target.closest("li[data-id]");
  if (!item) return;
  const id = Number(item.dataset.id);
  const task = tasks.find((t) => t.id === id);
  if (e.target.matches(".check")) task.done = !task.done;
  else if (e.target.matches(".edit")) return startEdit(task);
  else if (e.target.matches(".delete")) {
    if (!confirm("Delete this task?")) return;
    tasks = tasks.filter((t) => t.id !== id);
  }
  saveTasks(); render();
});

function escapeHtml(str) {
  return str.replace(/[<>"']/g, (c) => ({ "&": "&amp;",
    "<": "&lt;", ">": "&gt;", "'": "&quot;", '"': "&#39;" }[c]));
}
```

#### 7.1.7 Overdue Detection and Statistics

A task is treated as overdue when it is not completed and its due date is earlier than the current date. The current date is calculated in the user's local time zone and formatted as YYYY-MM-DD so that it can be compared directly with the stored due date. After every change, the statistics panel and progress bar are recalculated from the task list.

### Listing 7.7 Overdue detection and statistics

```
function today() {
  const d = new Date();
  d.setMinutes(d.getMinutes() - d.getTimezoneOffset());
  return d.toISOString().slice(0, 10);
}

const isOverdue = (t) => !t.done && t.due && t.due < today();

function updateStats() {
  const done = tasks.filter((t) => t.done).length;
  const pct = tasks.length
    ? Math.round((done / tasks.length) * 100) : 0;
  $("#statTotal").textContent = tasks.length;
  $("#statPending").textContent = tasks.length - done;
  $("#statDone").textContent = done;
  $("#statOverdue").textContent = tasks.filter(isOverdue).length;
  $("#progressBar").style.width = pct + "%";
  $("#progressText").textContent = pct + "% complete";
}
```

#### 7.1.8 Clearing Completed Tasks

Over time, the list can fill up with tasks that have already been completed. The Clear completed option at the bottom of the list removes all of them in one step. The function first counts the completed tasks and does nothing if there are none; otherwise it asks for confirmation, keeps only the pending tasks, and then saves and re-renders the list.

### Listing 7.8 Clearing completed tasks

```
$("#clearDone").addEventListener("click", () => {
  const count = tasks.filter((t) => t.done).length;
  if (!count) return;
  if (!confirm(`Remove ${count} completed task(s)?`)) return;
  tasks = tasks.filter((t) => !t.done);
  saveTasks(); render();
});
```

## 7.1.9 Responsive Design and Dark Mode

CSS custom properties (variables) are used for all colours. Dark mode is enabled by adding a dark class to the body element, which overrides these variables. Media queries rearrange the statistics cards into two columns and stack the toolbar controls vertically on screens narrower than 600 pixels.

### Listing 7.9 Theme variables and media query in style.css

```
:root {
  --bg: #f3f4f8; --card: #ffffff; --text: #1f2937;
  --primary: #4f6ef7;
  --high: #ef4444; --medium: #f59e0b; --low: #22a06b;
}
body.dark {
  --bg: #111318; --card: #1c1f26; --text: #e5e7eb;
}
.task { border-left: 4px solid var(--medium); }
.task.high { border-left-color: var(--high); }
.task.low { border-left-color: var(--low); }

@media (max-width: 600px) {
  .stats { grid-template-columns: repeat(2, 1fr); }
  .toolbar, .form-row { flex-direction: column; }
  button { width: 100%; }
}
```

The theme toggle button switches the dark class on the body element and saves the selected theme in localStorage under a separate key. When the page is opened again, the saved theme is read and applied during initialisation, before the list is rendered, so the user always sees the theme they last selected.

### Listing 7.10 Loading, applying and toggling the theme

```
function loadTheme() {
  try { return localStorage.getItem(THEME_KEY) || "light"; }
  catch (e) { return "light"; }
}
function applyTheme(theme) {
  document.body.classList.toggle("dark", theme === "dark");
}
$("#themeToggle").addEventListener("click", () => {
  const next = document.body.classList.contains("dark")
    ? "light" : "dark";
  applyTheme(next);
  try { localStorage.setItem(THEME_KEY, next); } catch (e) {}
});
```

The way each part of the interface changes between large and small screens is summarised in Table 7.1.

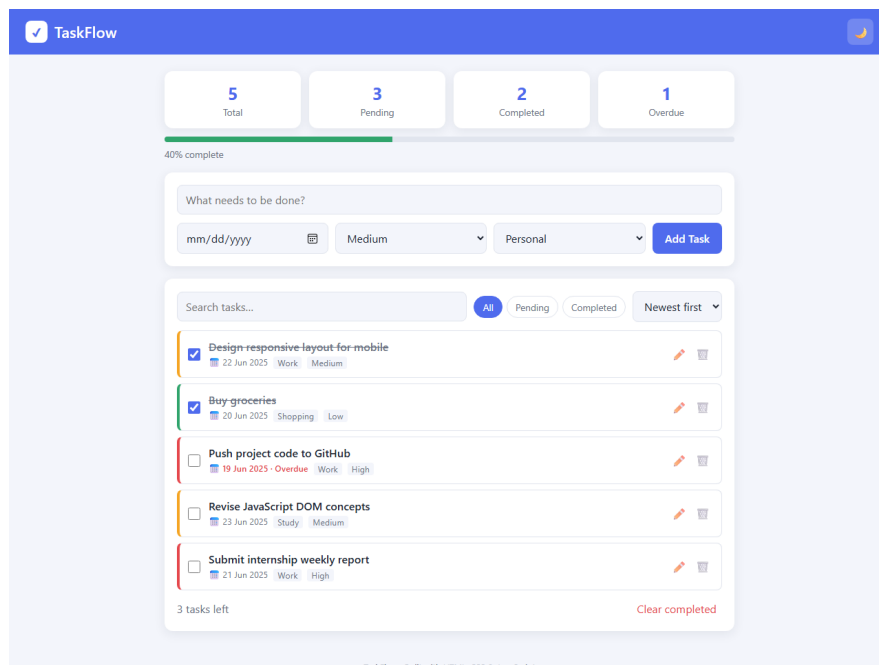
| Element          | Wider than 600 px                              | 600 px or Narrower                  |
|------------------|------------------------------------------------|-------------------------------------|
| Statistics cards | Four cards in a single row                     | Two cards per row                   |
| Task form        | Date, priority, category and button in one row | Fields wrap onto separate rows      |
| Toolbar          | Search, filters and sort in one row            | Controls stacked vertically         |
| Task items       | Title, tags and actions on one line            | Title and tags wrap onto more lines |
| Buttons          | Normal width                                   | Full width for easy tapping         |

*Table 7.1 Responsive Layout Behaviour*

## 7.2 Results

### 7.2.1 Home Screen

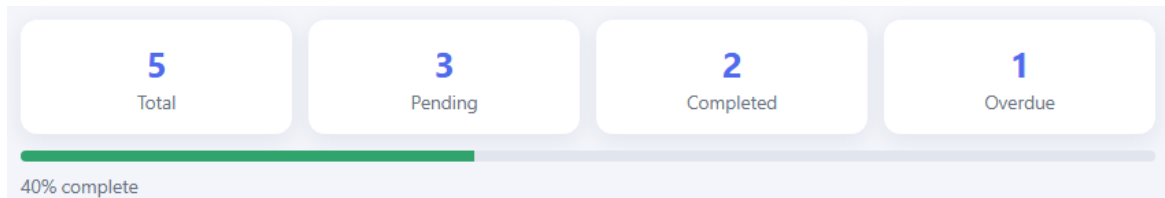
The main screen of TaskFlow in light mode on a desktop browser is shown in Figure 7.1. The statistics panel at the top shows the number of total, pending, completed and overdue tasks along with the progress bar. Completed tasks are struck through, overdue tasks are shown in red, and the coloured border on each task indicates its priority. In the example shown, two of the five tasks are completed, so the progress bar shows 40% completion, and the task “Push project code to GitHub” is marked as overdue because its due date has passed.



**Figure 7.1 TaskFlow Home Screen (Desktop, Light Mode)**

### 7.2.2 Statistics Panel and Progress Bar

Figure 7.2 shows the statistics panel. The four cards give the user an instant summary of the task list: 5 tasks in total, of which 3 are pending, 2 are completed and 1 is overdue. The green progress bar below the cards fills in proportion to the percentage of completed tasks, and the label underneath states the exact value. These numbers are recalculated by `updateStats()` after every action, so marking a task as completed immediately moves the progress bar forward.



**Figure 7.2 Statistics Panel and Progress Bar**

### 7.2.3 Task Entry Form

The task entry form, shown in Figure 7.3, is placed directly below the statistics panel. The title field displays the placeholder “What needs to be done?”, and the row below contains the date picker, the priority drop-down (defaulting to Medium), the category drop-down (defaulting to Personal) and the Add Task button. Using the browser’s built-in date picker ensures that the due date is always entered in a valid format.

What needs to be done?

mm/dd/yyyy Medium Personal **Add Task**

**Figure 7.3 Task Entry Form**

### 7.2.4 Search, Filter and Sort Toolbar

Figure 7.4 shows the toolbar above the task list. The search box filters tasks as the user types, the All, Pending and Completed buttons switch the status filter (the selected button is highlighted in blue), and the drop-down on the right changes the sort order between newest first, due date and priority. All three controls can be combined; for example, the user can view only pending Work tasks sorted by due date.

Search tasks... **All** Pending Completed Newest first

**Figure 7.4 Search, Filter and Sort Toolbar**

## 7.2.5 Task States and Priority Indicators

Figure 7.5 shows how different task states are displayed. The first two tasks are completed, so their checkboxes are ticked and their titles are struck through and greyed out. The third task is pending and its due date has passed, so the date is shown in red with the word “Overdue”. The colour of the left border shows the priority – orange for Medium, green for Low and red for High – and the category and priority are also shown as small tags below the title. The edit and delete buttons are placed on the right side of every task.

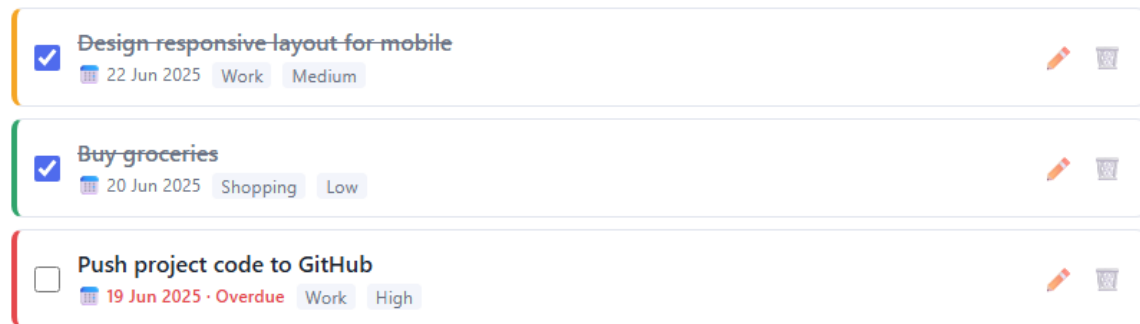


Figure 7.5 Task States and Priority Indicators

## 7.2.6 Dark Mode

Figure 7.6 shows the same screen in dark mode, which can be toggled using the button at the top-right corner. The selected theme is remembered across visits. Because all colours are defined as CSS variables, the dark theme required only the variables to be redefined and no change to the layout.

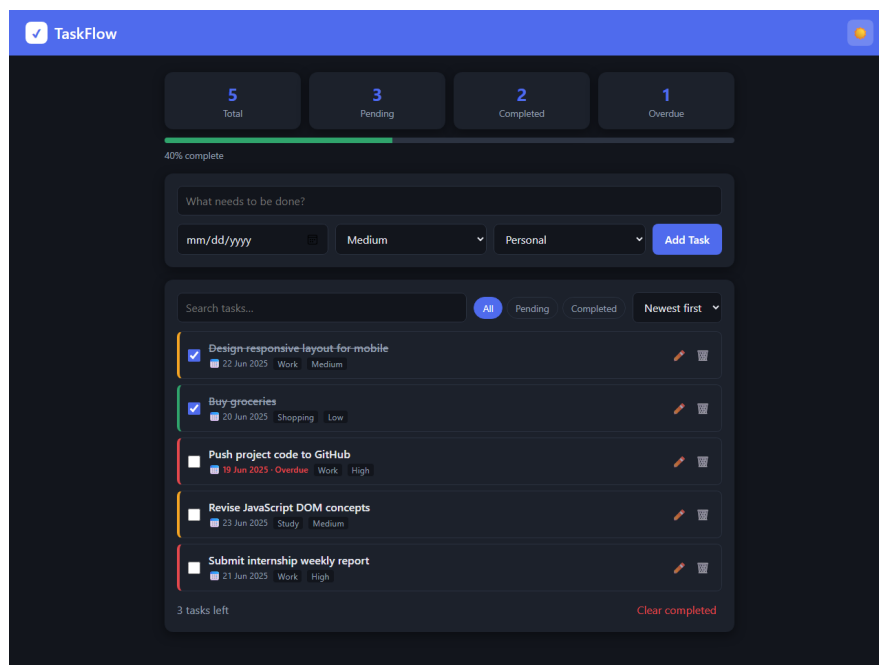
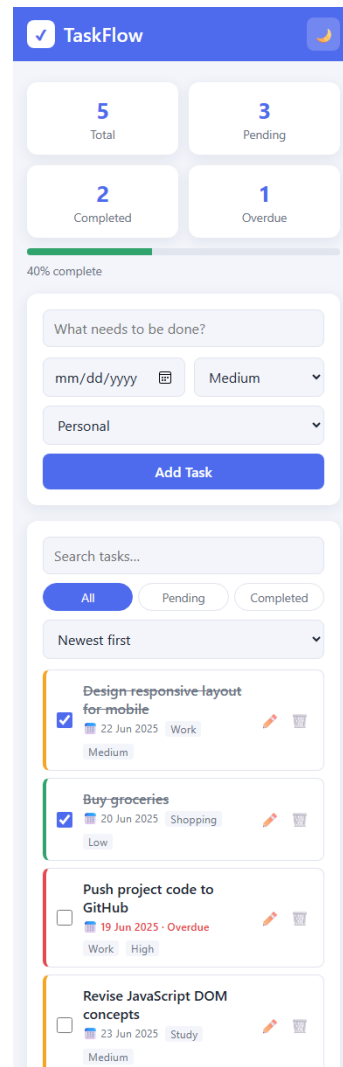


Figure 7.6 TaskFlow in Dark Mode

### 7.2.7 Mobile View

Figure 7.7 shows the application on a mobile screen. The layout automatically adapts to the narrow width: the statistics cards are arranged in two columns, the form fields and toolbar controls are stacked, and the buttons span the full width for easy tapping.



**Figure 7.7 TaskFlow on a Mobile Screen**

## 7.3 Testing

### 7.3.1 Testing Approach

The application was tested manually in Google Chrome and Microsoft Edge on desktop and in the mobile device emulation mode of the browser developer tools. Since TaskFlow is a front-end application driven by user interaction, black-box testing was used: each feature was exercised through the user interface and the actual behaviour was compared with the expected behaviour. The testing covered the following areas:

- **Functional testing:** Checking that every operation – adding, editing, completing, deleting, searching, filtering, sorting and clearing tasks – works as described in the functional requirements.

- **Validation testing:** Checking that invalid input, such as an empty title, is rejected with a clear message.
- **Persistence testing:** Refreshing the page to confirm that tasks and the theme preference are restored from localStorage.
- **Security testing:** Entering HTML tags in the task title to confirm that they are shown as plain text and not executed.
- **User interface and responsive testing:** Checking the layout in both themes and on a narrow mobile screen using device emulation.

### 7.3.2 Test Cases and Results

The test cases and their results are summarised in Table 7.2.

| TC | Test Case                      | Expected Result                         | Status |
|----|--------------------------------|-----------------------------------------|--------|
| 1  | Add a task with a title        | Task appears at the top of the list     | Pass   |
| 2  | Add a task with an empty title | Error message shown, task not added     | Pass   |
| 3  | Mark a task as completed       | Task struck through, statistics updated | Pass   |
| 4  | Edit an existing task          | Form filled, task updated on save       | Pass   |
| 5  | Delete a task                  | Confirmation shown, task removed        | Pass   |
| 6  | Task with past due date        | Marked as overdue in red                | Pass   |
| 7  | Filter by Pending / Completed  | Only matching tasks shown               | Pass   |
| 8  | Search by title or category    | List filtered as user types             | Pass   |
| 9  | Sort by due date and priority  | List reordered correctly                | Pass   |
| 10 | Clear completed tasks          | All completed tasks removed             | Pass   |
| 11 | Refresh the page               | Tasks restored from localStorage        | Pass   |
| 12 | Toggle dark mode and reload    | Theme applied and remembered            | Pass   |
| 13 | Enter HTML tags in title       | Shown as plain text, not executed       | Pass   |
| 14 | Filter with no matching tasks  | Empty-state message displayed           | Pass   |
| 15 | Open on 390 px mobile screen   | Layout adapts, no horizontal scroll     | Pass   |

*Table 7.2 Test Cases and Results*

All 15 test cases passed successfully, confirming that the application meets the objectives defined in Chapter 4.

### 7.3.3 Requirement Traceability

To make sure that every requirement identified in Chapter 6 was verified, each functional requirement was mapped to the test cases that cover it, as shown in Table 7.3. Every requirement is covered by at least one passing test case.

| Req. ID | Requirement (Summary)                       | Test Cases          | Status   |
|---------|---------------------------------------------|---------------------|----------|
| FR1     | Add a task with details                     | TC1                 | Verified |
| FR2     | Reject an empty title                       | TC2                 | Verified |
| FR3     | Edit, delete and complete tasks             | TC3, TC4, TC5, TC10 | Verified |
| FR4     | Search by title or category                 | TC8                 | Verified |
| FR5     | Filter by status and sort                   | TC7, TC9, TC14      | Verified |
| FR6     | Highlight overdue tasks and show statistics | TC3, TC6            | Verified |
| FR7     | Save and restore tasks                      | TC11                | Verified |
| FR8     | Switch between light and dark themes        | TC12                | Verified |

*Table 7.3 Requirement Traceability Matrix*

## 7.4 Challenges Faced and Solutions

Several problems were encountered during development. The main challenges and the way each of them was solved are summarised in Table 7.4.

| Challenge                                                                           | Solution                                                                       |
|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| Application stopped working when the stored data was invalid or storage was blocked | Wrapped loading and saving in try/catch blocks and fell back to sample tasks   |
| HTML tags typed in a task title were rendered as page elements                      | Escaped all user-entered text before inserting it into the page                |
| Adding a listener to every task button made the code repetitive                     | Used a single delegated click listener on the task list                        |
| Tasks due today were sometimes shown as overdue because of time zone differences    | Calculated today's date in local time and compared dates as YYYY-MM-DD strings |
| Toolbar and form controls overflowed horizontally on small screens                  | Used a media query to stack the controls vertically below 600 pixels           |
| Switching themes required changing many colour rules                                | Defined all colours as CSS variables and overrode them in a dark class         |

*Table 7.4 Challenges Faced and Solutions*

## **CHAPTER 8**

### **CONCLUSION**

#### **8.1 Conclusion**

The one-month Web Development internship at Vault of Codes provided valuable practical experience in building a complete web application from scratch. In this project, TaskFlow, a responsive task manager, was designed and developed using HTML5, CSS3 and JavaScript. The application allows users to add, edit, complete and delete tasks, assign due dates, priorities and categories, and search, filter and sort their tasks, while a statistics panel shows their progress and overdue tasks are highlighted automatically. Tasks are stored persistently in the browser using localStorage, and the interface works smoothly on both desktop and mobile screens with an optional dark mode. All the functional test cases were passed, showing that a useful and reliable productivity tool can be built using only core web technologies.

#### **8.2 Future Enhancements**

- Adding user authentication and a back-end database so that tasks can be synchronised across devices.
- Browser notifications and reminders for tasks that are due soon.
- Drag-and-drop reordering of tasks and support for sub-tasks.
- Converting the application into a Progressive Web App (PWA) so that it can be installed and used offline.
- Sharing task lists with other users for group projects.

#### **8.3 Learning Outcomes**

Through this internship, I gained practical knowledge of front-end web development, including semantic HTML, modern CSS layout techniques such as Flexbox and Grid, responsive design using media queries, and theming with CSS variables. I learned to manipulate the DOM, handle user events efficiently using event delegation, validate user input, and store data on the client side using the Web Storage API. I also became familiar with using browser developer tools for debugging and testing, and with Git and GitHub for version control. The internship improved my problem-solving, self-learning and documentation skills and gave me the confidence to build complete web applications independently.